1. Min-Max Search Algorithm for Chess Game

```
# Install once in the environment:
# pip install python-chess

import chess

PIECE_VALUES = {
    chess.PAWN: 100,
    chess.KNIGHT: 320,
    chess.BISHOP: 330,
    chess.ROOK: 500,
    chess.QUEEN: 900,
    chess.KING: 20000
}

def evaluate(board):
    if board.is_checkmate():
        return -100000 if board.turn == chess.WHITE else 100000
    if board.is_game_over():
        return 0

    score = 0
    for piece_type, value in PIECE_VALUES.items():
        score += len(board.pieces(piece_type, chess.WHITE)) * value
        score -= len(board.pieces(piece_type, chess.BLACK)) * value
    return score

def minimax(board, depth, maximizing):
    if depth == 0 or board.is_game_over():
        return evaluate(board), None

    best_move = None

    if maximizing:
        best_score = float("-inf")
        for move in board.legal_moves:
            board.push(move)
            score, _ = minimax(board, depth - 1, False)
            board.pop()

            if score > best_score:
                best_score = score
                best_move = move

        return best_score, best_move

    best_score = float("inf")
    for move in board.legal_moves:
        board.push(move)
        score, _ = minimax(board, depth - 1, True)
        board.pop()

        if score < best_score:
            best_score = score
            best_move = move

    return best_score, best_move

board = chess.Board()
score, move = minimax(board, 2, board.turn == chess.WHITE)

print("Best move:", move)
print("Evaluation:", score)
```

2. Alpha-Beta Pruning Algorithm for Chess Game

```
# Install once in the environment:
# pip install python-chess

import chess

PIECE_VALUES = {
    chess.PAWN: 100,
    chess.KNIGHT: 320,
    chess.BISHOP: 330,
    chess.ROOK: 500,
    chess.QUEEN: 900,
    chess.KING: 20000
}

def evaluate(board):
    if board.is_checkmate():
        return -100000 if board.turn == chess.WHITE else 100000
    if board.is_game_over():
```

```
        return 0

    score = 0
    for piece_type, value in PIECE_VALUES.items():
        score += len(board.pieces(piece_type, chess.WHITE)) * value
        score -= len(board.pieces(piece_type, chess.BLACK)) * value
    return score

def alpha_beta(board, depth, alpha, beta, maximizing):
    if depth == 0 or board.is_game_over():
        return evaluate(board), None

    best_move = None

    if maximizing:
        best_score = float("-inf")
        for move in board.legal_moves:
            board.push(move)
            score, _ = alpha_beta(
                board, depth - 1, alpha, beta, False
            )
            board.pop()

            if score > best_score:
                best_score = score
                best_move = move

            alpha = max(alpha, best_score)
            if beta <= alpha:
                break

        return best_score, best_move

    best_score = float("inf")
    for move in board.legal_moves:
        board.push(move)
        score, _ = alpha_beta(
            board, depth - 1, alpha, beta, True
        )
        board.pop()

        if score < best_score:
            best_score = score
            best_move = move

        beta = min(beta, best_score)
        if beta <= alpha:
            break

    return best_score, best_move

board = chess.Board()
score, move = alpha_beta(
    board, 3, float("-inf"), float("inf"),
    board.turn == chess.WHITE
)

print("Best move:", move)
print("Evaluation:", score)
```

3. Min-Max Search Algorithm for Tic-Tac-Toe Game

```
# Tic-tac-toe using the Min-Max algorithm.

HUMAN = "X"
AI = "O"
EMPTY = " "

board = [EMPTY] * 9

WINNING_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8),
    (0, 3, 6), (1, 4, 7), (2, 5, 8),
    (0, 4, 8), (2, 4, 6)
]

def winner(state):
    for a, b, c in WINNING_LINES:
        if state[a] == state[b] == state[c] != EMPTY:
            return state[a]
    return None

def minimax(state, is_maximizing):
    result = winner(state)

    if result == AI:
```

```
        return 1
    if result == HUMAN:
        return -1
    if EMPTY not in state:
        return 0

    if is_maximizing:
        best_score = float("-inf")
        for i in range(9):
            if state[i] == EMPTY:
                state[i] = AI
                score = minimax(state, False)
                state[i] = EMPTY
                best_score = max(best_score, score)
        return best_score

    best_score = float("inf")
    for i in range(9):
        if state[i] == EMPTY:
            state[i] = HUMAN
            score = minimax(state, True)
            state[i] = EMPTY
            best_score = min(best_score, score)
    return best_score

def best_ai_move(state):
    best_score = float("-inf")
    move = None

    for i in range(9):
        if state[i] == EMPTY:
            state[i] = AI
            score = minimax(state, False)
            state[i] = EMPTY

            if score > best_score:
                best_score = score
                move = i

    return move

def display():
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i + 3]))
        if i < 6:
            print("--+---+--")

print("Positions: 0 to 8")
display()

while True:
    try:
        human_move = int(input("Enter your move (0-8): "))
    except ValueError:
        print("Enter a valid number.")
        continue

    if human_move < 0 or human_move > 8 or board[human_move] != EMPTY:
        print("Invalid move. Try again.")
        continue

    board[human_move] = HUMAN

    if winner(board) == HUMAN:
        display()
        print("You win!")
        break

    if EMPTY not in board:
        display()
        print("Draw!")
        break

    ai_move = best_ai_move(board)
    board[ai_move] = AI
    print("AI chooses:", ai_move)
    display()

    if winner(board) == AI:
        print("AI wins!")
        break

    if EMPTY not in board:
        print("Draw!")
        break
```